

**THE UNIVERSITY OF AZAD JAMMU AND
KASHMIR, MUZAFFARABAD**



COURSE TITLE

Object Oriented Programming

COURSE CODE

CS-1204

PROJECT TITLE

Personal Productivity App

STUDENT NAME

**Shahzad Ahmed Awan, Sughra
Mumshad, Maryam Shaheen**

STUDENT ROLL NUMBER

2024-SE-13,15,30

INSTRUCTOR NAME

Engr. Awais Rathore

SUBMISSION DATE

26th September, 2025

Title

OOP Semester Project

Productivity App — Professional Project Report

Project: Personal Productivity App (C++ / OOP / File-based persistence)

Date: 2025-09-26

Prepared for: OOP Semester Project

Prepared by: Group 01

1. Introduction

This project is a feature-rich productivity application built entirely in C++. It integrates multiple life management modules into a single console-based system, offering tools for fitness tracking, study planning, financial management, habit tracking, and mental wellness.

The goal of the application is to provide a holistic productivity suite that enables users to balance health, academics, finances, personal growth, and mental well-being.

2. Objectives

- Provide an all-in-one solution for productivity enhancement.
- Ensure user-friendliness through structured menus.
- Enable data persistence with file handling.
- Promote healthy lifestyle and study habits.
- Offer extendable modular design for future upgrades.

3. Features Overview

Module	Core Functionalities
Fitness Tracker	Track workouts, log water intake, monitor steps, calculate calories, and save history.
Study Planner	Manage subjects, track study hours, maintain deadlines, and calculate study progress.
Finance Manager	Track income/expenses, manage budgets, savings goals, and generate financial summaries.
Habit Tracker	Define custom habits, log daily completions, calculate streaks, and view progress.
Mental Wellness	Mood logging, relaxation tips, stress management, and reminders for self-care.
User System	Profile creation with basic information, centralizing access to all features.

4. Project Architecture

The system is built using an object-oriented modular design to ensure clarity, scalability, and maintainability.

- **Encapsulated Modules** – Each feature is implemented as a separate class (`FitnessTracker`, `FinanceManager`, `HabitTracker`, `StudyManager`, `MentalWellness`).
- **Central User Class** – The class integrates all modules, maintaining a unified profile and managing personalized data.
- **Main Controller** – The `main.cpp` file acts as the entry point, providing a menu-driven interface and delegating control to modules.
- **Persistence Layer** – File handling is used to store user progress and reload it in future sessions.

4.1 UML Class Diagram

The UML diagram highlights the modular design:

- `User` acts as the central hub.
- Each tracker or manager is modeled as a separate class.
- Low coupling and high cohesion make the design easier to maintain and extend.

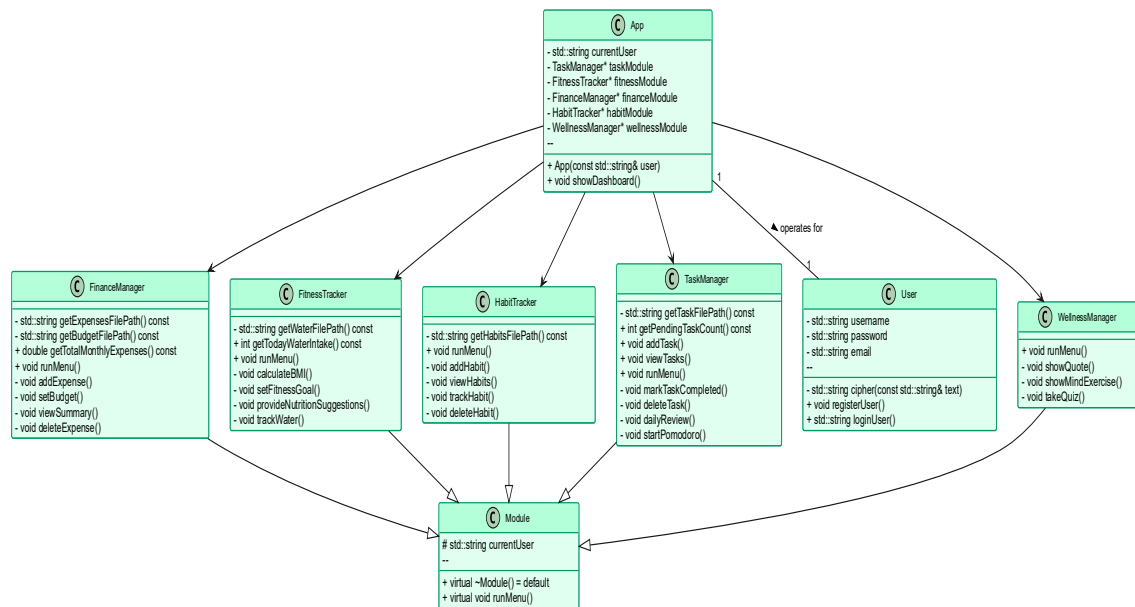


Figure 1: UML Class Diagram representing System Static Structure

4.2 User Flow

The flow of the application is straightforward and user-friendly:

- The program starts with a welcome screen where the user can log in or register.
- After successful login, the main menu is displayed.
- From the main menu, the user can navigate to one of the available modules:
 - Fitness Tracker
 - Finance Manager

- Habit Tracker
- Study Task Manager
- Mental Wellness
- Inside each module, the user can perform specific actions (e.g., add tasks, track fitness, manage expenses).
- Whenever an action is completed, data is saved automatically to ensure

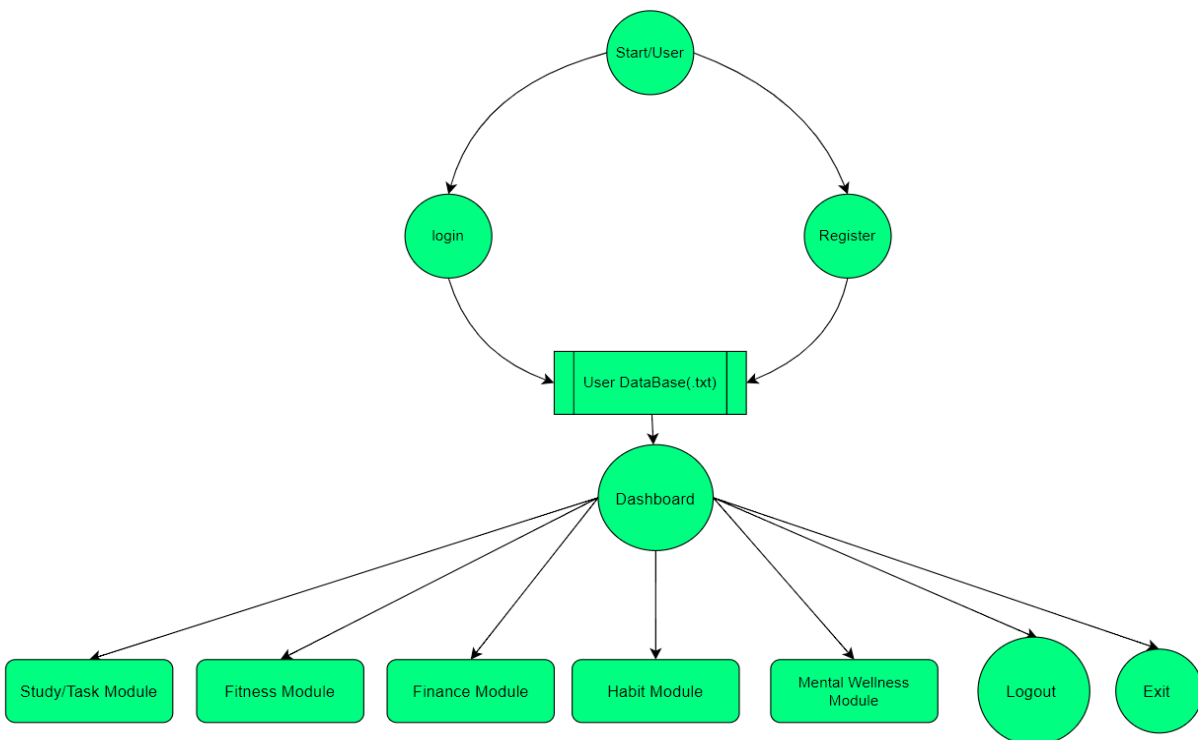


Figure 2: User Activity / Flow Diagram

progress is not lost.

- Finally, the user can choose to exit, and the system ends the session while preserving all data for the next use.

5. Data Persistence

- **File Handling:** Each module maintains logs (text files) for continuity.
- **User Profiles:** Store and reload personalized user data.
- **Reports:** Summarized in readable format for progress visualization.

6. Project File Structure

File	Purpose
<code>main.cpp</code>	Entry point, integrates all modules and handles the main menu.
<code>User.h</code> / <code>User.cpp</code>	Defines the User profile and bridges modules.
<code>FitnessTracker.*</code>	Implements workout, water intake, calories, and activity tracking.
<code>StudyPlanner.*</code>	Manages study plans, deadlines, and subject-wise progress.
<code>FinanceManager.*</code>	Handles income/expense logging, savings, and financial goals.
<code>HabitTracker.*</code>	Defines habits, tracks streaks, and logs daily progress.
<code>MentalWellness.*</code>	Provides mood logging, stress relief exercises, and wellness monitoring.
<code>utils.h</code>	Utility functions supporting input handling, validation, and formatting.

7. Appendix — File Index & Data Formats

The system uses simple text files for persistence. Each file is dedicated to a specific feature or user profile, ensuring modular and personalized data storage.

File Index

- **users.txt** – User registry for login and authentication.
- **quotes.txt** – Motivational quotes for the wellness module.
- **quiz_questions.txt** – Quiz questions with multiple-choice options.
- **exercises.txt** – Fitness routines and workouts.
- **study_tasks_<user>.txt** – Study tasks per user.
- **habits_<user>.txt** – Daily habits per user.
- **fitness_goals_<user>.txt** – User fitness goals and progress.
- **water_<user>.txt** – Daily water intake records.
- **budget_<user>.txt** – User budget data.
- **expenses_<user>.txt** – Expense logs per user.
- **daily_review_<user>_<date>.txt** – Daily reflections and wellness notes.

Data Format Examples

- **study_tasks_ali.txt** → shahzad,1,22,12,2025,1
(*task_name, priority, day, month, year, status*)

- **water_ali.txt** → 2025-09-26,6
(*date, glasses_of_water*)
- **quiz_questions.txt** → What_is_the_capital_of_France?,Paris,Lahore,Berlin
(*question, correct_answer, option2, option3, option4*)
- **quotes.txt** → "The only way to do great work is to love what you do."

8. Conclusion

The project successfully integrates study, habit, fitness, finance, and wellness into one modular C++ system. It demonstrates strong design principles, realistic features, and extensibility, making it suitable for academic and professional evaluation.